CourtAccess Form Assistant — Integration Handoff Document

Author: Mohit Sharma
Date: March 2026
Standalone repo: `courtaccess-form-assistant`
Target repo: `SunnyYadav16/court-access-ai`

---

1. What This Feature Does

The Form Assistant is a Gemini-powered guided form-filling tool. Users select a Massachusetts court form, then answer questions one at a time via text or voice (in English, Spanish, or Portuguese). Answers are validated and formatted in real-time, then written directly into the PDF's AcroForm fields for download.

This is a standalone feature — it does not modify the speech interpretation pipeline or the document translation pipeline.

---

2. Standalone Repo — Current State

What is complete and tested

- ✅ Guided fill conversation loop (text) — end-to-end validated on `complaint_for_partition.pdf` (6 pages, 169 fields)
- ✅ AcroForm PDF export with correct field writing via `pdfrw`
- ✅ Flat PDF export via `reportlab` overlay (used for `petition_209a`)
- ✅ Automatic form schema generation via `introspect_form.py` (PyMuPDF + Gemini enrichment)
- ✅ Field highlight overlay on PDF canvas, multipage, correct coordinates
- ✅ Input validation in Gemini system prompt: dates, phones, emails, state abbreviations, checkboxes, dropdowns
- ✅ Null/skip handling — skipped and signature fields are excluded from export
- ✅ Dynamic form catalog — add a PDF + run `introspect_form.py`, form appears automatically
- ✅ RAG pipeline — 201 chunks across 3 forms (3 form-level + 198 field-level), three-strategy retrieval: form overview + current field + cross-field semantic search on questions

What is not complete

- GCS integration — form catalog reads from local `corpus/instructions/*.json`, not GCS

- Voice TTS uses `gemini-2.5-flash-preview-tts` which has intermittent 500 errors — a 4-voice retry loop is in place (`Kore` → `Puck` → `Aoede` → `Charon`) that resolves most failures automatically

---

3. RAG Pipeline

What it does and why

The assistant uses Retrieval-Augmented Generation (RAG) to ground Gemini's field instructions in official source material scraped from mass.gov — rather than relying on Gemini's training knowledge about Massachusetts court forms, which may be stale or incorrect for obscure fields.

For well-known fields like `case_name` or `date`, this difference is small. For fields like `proportional_interest_percentage_or_fraction_row1` on the partition form, Gemini's training data is sparse and RAG is the only reliable source of correct instructions.

Architecture

Each field in each form is stored as one chunk in ChromaDB (field-level chunking, not page-level). Page-level chunks pollute retrieved context with unrelated field instructions — one chunk per field is the correct granularity.

```
corpus/instructions/*.json
        ↓ build_index.py
ChromaDB (chroma_db/)
        ↓ retriever.py (called per message turn)
gemini_client.py → injects RETRIEVED CONTEXT above system prompt rules
```

Files

File	Purpose
`scripts/build_index.py`	Reads all JSON schemas, embeds field instructions via `gemini-embedding-001`, stores in ChromaDB
`services/retriever.py`	Exact lookup by `form_id::field_name` first; semantic fallback if not found
`chroma_db/`	Local ChromaDB storage — add to `.gitignore`, rebuild from corpus on each deployment

Running the indexer

```powershell
# Index all forms
python -m scripts.build_index

# Index one specific form (after adding a new one)
python -m scripts.build_index complaint_for_partition
```

Current index: **198 chunks** across 3 forms (169 complaint_for_partition, 18 notice_of_appearance, 12 petition_209a).

Graceful degradation

`is_index_ready()` is checked before every retrieval call. If ChromaDB is empty or missing, the system falls back to the existing behavior — full form text passed directly to Gemini — with no errors and no broken sessions.

MLOps integration

The indexer should be added as a final step in the `generate_schema` Airflow task (see Section 7), so new forms are automatically indexed when scraped:

```python
# After uploading schema JSON to GCS:
from scripts.build_index import build_index
build_index(target_form_id=form_id)
```

---

4. File Inventory — What to Copy Into the Main Repo

Backend files (copy into `court-access-ai/backend/`)

File	Destination	Notes
`routers/assistant.py`	`backend/routers/assistant.py`	All REST endpoint Register in `main.` (see Section 6).
`routers/voice_ws.py`	`backend/routers/voice_ws.py`	Voice endpoints. Register only if voi being activated.

File	Destination	Notes
`services/gemini_client.py`	`backend/services/gemini_client.py`	Builds system pron calls Gemini, parse response. Calls ret on every turn.
`services/field_extractor.py`	`backend/services/field_extractor.py`	Reads field schema JSON corpus files.
`services/pdf_filler.py`	`backend/services/pdf_filler.py`	AcroForm fill via `pdfrw`; overlay fil `reportlab`+`Pyl`
`services/reply_parser.py`	`backend/services/reply_parser.py`	Strips `FIELD_UPD` from Gemini reply; normalises nulls.
`services/retriever.py`	`backend/services/retriever.py`	ChromaDB lookup RAG. Exact match semantic fallback.
`services/form_catalog.py`	`backend/services/form_catalog.py`	**REPLACE** with GC aware version in Ph (see Section 7).
`scripts/introspect_form.py`	`backend/scripts/introspect_form.py`	Generates JSON schema from a PDF once per new form
`scripts/build_index.py`	`backend/scripts/build_index.py`	Builds ChromaDB vector index from corpus JSON files.
`corpus/instructions/*.json`	`backend/corpus/instructions/`	Schema files for all forms currently supported.
`chroma_db/`	`backend/chroma_db/`	ChromaDB storage to `.gitignore` – rebuild on each deployment via `build_index.py`
`forms/*.pdf`	`backend/forms/`	PDFs for local dev Production uses GC

Frontend files (copy into `court-access-ai/frontend/src/`)

File	Destination	Notes
`components/ChatInterface.tsx`	`frontend/src/components/ChatInterface.tsx`	Chat UI, r handling, managen
`components/PdfViewer.tsx`	`frontend/src/components/PdfViewer.tsx`	Multipag renderer highlight
`components/VoiceButton.tsx`	`frontend/src/components/VoiceButton.tsx`	Voice inp button.
`api.ts`	Merge into existing `frontend/src/api.ts`	Add `Fie` `Sessior` `Message` interfaces assistant functions overwrite API funct

New dependencies to add

Backend (`requirements.txt`):

```
google-genai>=1.0.0
pymupdf>=1.27.0
pdfrw>=0.4
reportlab>=4.0.0
PyPDF2>=3.0.0
pdfplumber>=0.10.0
chromadb>=0.5.0        # for RAG pipeline when built
```

Frontend (`package.json`):

```
pdfjs-dist
```

---

4. Critical Rules — Do Not Change These

These rules exist for correctness and safety reasons. Changing them will break the system.

Rule	Location	Why it must not change
`role: "model"` in conversation history	`ChatInterface.tsx` history mapping	Gemini SDK rejects `"assistant"` as a role value
Null answers filtered before export	`assistant.py` export endpoint	Passing null to `pdfrw` crashes the PDF writer
`api.ts` interfaces frozen	`api.ts`	Frontend and backend are coupled on response shapes; changing one breaks the other
Scope lock in system prompt	`gemini_client.py` rules 1–2	Prevents Gemini from giving legal advice; legal liability risk if removed
Field update keys validated against session schema	`assistant.py` `_validated_field_updates`	Prevents Gemini hallucinating field names that don't exist on the form
`FIELD_UPDATE` parsed from first line	`reply_parser.py` uses `find` not `rfind`	Gemini emits update on line 1; `rfind` would return empty reply text

6. Registering the Router in `main.py`

In the main repo's `backend/main.py`, add:

```python
from routers.assistant import router as assistant_router
# existing routers:
# app.include_router(speech_router, prefix="/api")
# app.include_router(documents_router, prefix="/api")

# Add this:
app.include_router(assistant_router, prefix="/api")
```

The form assistant uses these endpoints — all prefixed `/api`:

```
GET  /api/forms
GET  /api/forms/{form_id}/pdf
POST /api/session
POST /api/assistant/message
POST /api/assistant/export
```

None of these conflict with existing routes in the main repo.

---

7. Phase 2 — GCS Integration (Replace Local File Catalog)

The current `form_catalog.py` reads schema JSONs from `corpus/instructions/*.json` on disk. In production, replace it with this pattern:

python

```python
# backend/services/form_catalog.py (GCS version)
from google.cloud import storage

def load_form_catalog() -> dict[str, dict]:
    client = storage.Client()
    bucket = client.bucket(os.getenv("GCS_BUCKET"))

    catalog = {}
    # Query PostgreSQL form_catalog table for all active forms
    # For each form, download schema JSON from GCS
    # gs://bucket/schemas/{form_id}.json
    for row in db.query("SELECT * FROM form_catalog WHERE status = 'active'"):
        blob = bucket.blob(f"schemas/{row.form_id}.json")
        schema = json.loads(blob.download_as_text())
        catalog[row.form_id] = {
            "name": schema["form_name"],
            "pdf_gcs_path": f"forms/original/{row.form_id}.pdf",
            "fill_mode": schema.get("pdf_fill_mode", "overlay"),
        }
    return catalog
```

The rest of the codebase calls `get_form_catalog()` and `get_form_entry()` — those interfaces stay identical. Only the data source changes.

---

8. Airflow DAG Integration

Add one task to the end of `form_pretranslation_dag`:

python

```python
from scripts.introspect_form import introspect

def generate_and_upload_schema(form_id: str, local_pdf_path: str, **context):
    """
    Run field introspection on the PDF and upload result to GCS.
    Called after ES + PT translations are written to GCS.
    """
    import tempfile, json
    from google.cloud import storage

    # Run introspection (outputs JSON to corpus/instructions/{form_id}.json locally)
    introspect(local_pdf_path, form_id)

    # Upload schema to GCS
    schema_path = f"corpus/instructions/{form_id}.json"
    client = storage.Client()
    bucket = client.bucket(os.getenv("GCS_BUCKET"))
    blob = bucket.blob(f"schemas/{form_id}.json")
    blob.upload_from_filename(schema_path)

    # Update form_catalog in PostgreSQL
    db.execute(
        "UPDATE form_catalog SET schema_gcs_path = %s, schema_generated_at = NOW() W
        (f"schemas/{form_id}.json", form_id)
    )

    # Rebuild RAG index for this form
    from scripts.build_index import build_index
    build_index(target_form_id=form_id)

generate_schema = PythonOperator(
    task_id="generate_form_schema",
    python_callable=generate_and_upload_schema,
    op_kwargs={
        "form_id": "{{ dag_run.conf['form_id'] }}",
        "local_pdf_path": "{{ dag_run.conf['local_pdf_path'] }}"
    },
    dag=dag,
)

# Append to existing chain:
# legal_review_pt >> generate_schema
```

This runs once per new or updated form, after translations are complete. Schema is then available to the form assistant immediately.

---

9. Frontend Integration

The form assistant should appear as a third navigation option on the home screen, alongside "Real-Time Translation" and "Upload Document":

```tsx
{ icon: "📋", title: "Fill a Court Form", desc: "Answer questions to fill Massachuse
```

This routes to the form selection screen (the `screen === "select"` branch in `App.tsx`), which then loads the chat + PDF viewer layout.

No changes needed to existing frontend screens.

---

10. Environment Variables Required

Add to `.env` / `.env.example` in the main repo:

```
GEMINI_API_KEY=your_key_here          # from aistudio.google.com
FORM_TEXT_MAX_CHARS=48000             # optional, default 48000
```

`GEMINI_API_KEY` is the only hard requirement. The form assistant uses `gemini-2.5-flash` and `google-genai` SDK (not `google-generativeai` — different package, different import path).

---

11. How to Add a New Form (Post-Integration)

1. PDF is downloaded to GCS by the scraper DAG automatically
2. `form_pretranslation_dag` translates it and triggers `generate_schema` task
3. Schema JSON is written to `gs://bucket/schemas/{form_id}.json`
4. Form appears automatically in `GET /api/forms` — no code changes needed

For local development:

```powershell
```

```
# Copy PDF to backend/forms/
# Then run introspection and indexing:
python -m scripts.introspect_form forms/your_form.pdf your_form_id
python -m scripts.build_index your_form_id
# Restart backend — form appears in /api/forms automatically
```